

Report of

**An Interpretability Extension for YouTube Comment
Sentiment Analysis Using TF-IDF Keywords and Bigrams**

Table of Contents

1.	Introduction.....	2
2.	Background and Original Paper Summary	3
3.	Original Paper Components.....	4
4.	Original Paper Workflow / Mental Model.....	6
5.	Original Method Pipeline.....	11
6.	Original Experiments Summary	14
7.	Assumptions and Limitations	16
8.	Critical Analysis and Selected Gap	18
9.	Proposed Extension	24
10.	Implementation - Our Work	26
11.	Results - Our Work	31
12.	Discussion	41
13.	Conclusion.....	44
14.	References	45
15.	Appendix	45

1. Introduction

YouTube has become one of the most widely used platforms for publishing and watching video content. Viewers often express their opinions through comments, and these comments can provide useful feedback for content creators. However, manually reading and understanding a large number of comments is difficult and time-consuming. Because of this, sentiment analysis can be used to automatically identify whether comments are positive, neutral, or negative.

The selected research paper, “Evaluating Machine Learning Models for Sentiment Analysis of YouTube Comments and Creating an Accessible Web Application for Comment Analysis,” focuses on machine-learning-based sentiment analysis of YouTube comments. The paper compares several machine learning models, including XGBoost, Random Forest, Logistic Regression, Gradient Boosted Decision Tree, AdaBoost, and Bagging Classifier. The goal of the original paper is to classify YouTube comments into positive, neutral, and negative sentiment categories and deploy the selected model in a user-friendly web application.

The original paper is useful because it provides a practical system for YouTube comment sentiment classification. However, sentiment labels alone do not fully explain what users are actually talking about inside each sentiment group. For example, knowing that a comment is positive or negative is helpful, but it is also useful to know which words or phrases are common in positive, neutral, and negative comments.

Therefore, this project extends the original paper by adding an interpretability layer. The selected extension is “TF-IDF Keyword Summary + Bigram Analysis for Each Sentiment Group.” In this extension, comments are grouped into positive, neutral, and negative categories. Then, TF-IDF keyword extraction and bigram analysis are applied separately to each sentiment group. This helps identify important words and common two-word phrases inside positive, neutral, and negative comments.

The main purpose of this project is not to replace the original paper’s sentiment classification system. Instead, the purpose is to make the sentiment analysis more explainable. By adding keyword and bigram summaries, content creators can understand not only whether audience feedback is positive, neutral, or negative, but also what topics or phrases appear inside those sentiment groups.

2. Background and Original Paper Summary

2.1 Plain-English Summary

This paper is about analyzing YouTube comments to decide whether they are positive, negative, or neutral. The authors say this is useful because content creators need feedback on how people respond to their videos, and doing this manually is difficult because there are so many comments.

The study compares several machine learning models with NLP techniques for sentiment analysis of YouTube comments. The authors say they trained and tested the models using a Kaggle dataset, and that the bagging classifier performed best, reaching 95.8% accuracy. They then used that model in a user-friendly web application for analyzing live and online YouTube video comments.

2.2 Key Points

I. Problem:

YouTube creators need a way to understand whether comments on their videos are positive, negative, or neutral.

II. Why it is hard:

There are many YouTube comments, and they can vary a lot in wording and meaning. Manually reading and classifying them is very challenging.

III. Main idea:

Use NLP techniques and machine learning models to classify YouTube comments into positive, negative, and neutral categories, then deploy the best model in a web application.

IV. What they claim to show:

They claim that among the tested models, the bagging classifier performed best, with 95.8% accuracy. They also claim that the model was deployed into a user-friendly web interface for analyzing YouTube comments.

2.3 Unclear Points

- a) The paper does not specify state whether the analyzed “live YouTube comments” are truly real-time streaming comments or comments collected from online videos.
- b) The paper does not specify explain what features or NLP preprocessing steps were used.
- c) The paper does not specify describe the Kaggle dataset, such as its size, language, labels, or source.
- d) The paper does not specify state how the train/test split or validation was done.
- e) The paper does not specify explain how the web application connects to YouTube comments.

3. Original Paper Components

3.1 List of Components

- **ID:** P0
Name: Problem and Motivation
Section(s): Abstract, 1. Introduction, 1.1 Objectives
Description: The paper focuses on sentiment analysis of YouTube comments. The problem is that content creators need feedback from comments, but manually analyzing large and varied comment sets is difficult. The goal is to classify comments as positive, negative, or neutral.
- **ID:** P1
Name: Research Context and Prior Work
Section(s): 2. Related work, 2.1 Literature review
Description: This part describes earlier work on YouTube comment analysis, sentiment analysis, hate speech detection, and machine-learning-based classification. It lists methods used in prior studies, including Naive Bayes, SVM, Logistic Regression, LSTM, Transformer Network, GRU, CNN, and others. The paper identifies gaps such as the need for more accurate and faster models and a user- friendly application.
- **ID:** P2
Name: Dataset Setup
Section(s): 3. Dataset
Description: The paper uses a publicly available Kaggle dataset. The dataset contains YouTube comments scraped using Google's API Key and labels them as negative, neutral, or positive using values 0, 1, and 2. The dataset is described as imbalanced because the sentiment classes are unevenly distributed.
- **ID:** P3
Name: Proposed System Workflow
Section(s): 4. Proposed System, Figure 2 Workflow Description
Description: The proposed system takes live YouTube comments and applies NLP techniques and machine learning models. It classifies comments into positive, neutral, and negative categories. The final output is shown through a web or mobile app interface for YouTube content creators.
- **ID:** P4
Name: Text Preprocessing
Section(s): 4.1 Data Preprocessing using NLP techniques, 4.1.1 Text processing
Description: This component prepares raw comments before model training. The steps

include tokenization, lowercasing, stopword removal, and cleaning punctuation, special characters, and numbers. The purpose is to convert messy text into a more standard form for analysis.

- **ID:** P5
Name: Feature Extraction and Text Analysis
Section(s): 4.2 Feature extraction, 4.2.1 Text analysis
Description: This part converts text into numerical features that machine learning models can use. The paper mentions TF-IDF, Bag-of-Words, and Word2Vec. It also mentions NLTK for NLP tasks such as tokenization, stemming, tagging, text classification, and parsing.
- **ID:** P6
Name: Machine Learning Models
Section(s): 4.3 ML models
Description: The paper compares six machine learning models: XGBoost, Random Forest, Logistic Regression, Gradient Boosted Decision Tree, AdaBoost, and Bagging Classifier. The Bagging Classifier is described as helping improve robustness and accuracy by reducing overfitting. The best model is later used in the web application.
- **ID:** P7
Name: Web Application
Section(s): 5. Web Application, 7.2 Working of the Developed Web App
Description: The web application uses the deployed Bagging Classifier model. It allows users to enter a YouTube video URL and then analyzes related comments as positive, negative, or neutral. The app displays results such as tabular data, sentiment distribution, and changes over time.
- **ID:** EXP
Name: Experimental Evaluation
Section(s): 5.1 Experimental work, 6. Performance metrics, 7. Results and Discussion, 7.1 Performance analysis
Description: The experiments were carried out using Python, NLTK libraries, TextBlob, VADER, and Google Colab. The paper evaluates models using accuracy, precision, recall, and F1-score. The reported results say the Bagging Classifier has the highest accuracy at 95.82%, while Gradient Boosted Decision Tree has the highest precision at 97.98%.
- **ID:** C0
Name: Conclusion and Future Work
Section(s): 8. Conclusion

Description: The paper concludes that the Bagging Classifier performed best among the tested models for YouTube comment sentiment analysis. It also says the authors created a user-friendly web interface for live and online YouTube comment analysis. Future work includes ensemble learning for multilingual contexts and better handling of imbalanced datasets.

3.2 Missing/Unclear Components

- **Exact dataset details:** Dataset size, language, number of comments per class, and exact Kaggle dataset name are not clear from provided text.
- **Train/test setup:** The train-test split, validation method, and whether cross-validation was used are not clear from provided text.
- **Real-time/live setup:** The paper says “live YouTube comments,” but the exact way live comments are collected and processed is not clear from provided text.
- **Feature choice actually used:** The paper mentions TF-IDF, Bag-of-Words, and Word2Vec, but which one was finally used for each experiment is not clear from provided text.
- **Preprocessing details:** The paper lists basic preprocessing steps, but exact implementation details are not clear from provided text.
- **Web app implementation details:** The framework, backend/frontend tools, deployment method, and API handling are not clear from provided text.
- **Handling class imbalance:** The dataset is described as imbalanced, but the method used to handle imbalance is not clear from provided text.
- **Reason for conclusion mismatch:** The results section reports Bagging Classifier accuracy as 95.82%, while the conclusion snippet says about 95.88%. The reason for this difference is not clear from provided text.

4. Original Paper Workflow / Mental Model

4.1 Component Dependency Explanation

- i. **ID:** P0
Depends on: []
Used by: [P1, P2, P3, P6, P7, EXP, C0]

Short note: This is the root component. The rest of the paper is built around the problem of classifying YouTube comments as positive, negative, or neutral.

ii. **ID:** P1

Depends on: [P0]

Used by: [P3, P6, P7, EXP, C0]

Short note: The related work depends on the problem because it reviews earlier attempts to solve similar sentiment-analysis tasks. It also motivates the proposed system, the model comparison, and the need for a user-friendly app.

iii. **ID:** P2

Depends on: [P0]

Used by: [P4, P5, P6, EXP, C0]

Short note: The dataset setup depends on the problem because the task needs labeled YouTube comments. The dataset is then used for preprocessing, feature extraction, model training, and evaluation.

iv. **ID:** P3

Depends on: [P0, P1, P2]

Used by: [P4, P5, P6, P7, EXP, C0]

Short note: The proposed workflow depends on the problem, the gaps found in prior work, and the availability of comment data. It gives the overall pipeline that the later technical components follow.

v. **ID:** P4

Depends on: [P2, P3]

Used by: [P5, P6, EXP, P7, C0]

Short note: Text preprocessing depends on having raw comment data and a system pipeline. Its output is needed before feature extraction and model training can happen.

vi. **ID:** P5

Depends on: [P2, P3, P4]

Used by: [P6, EXP, P7, C0]

Short note: Feature extraction depends on cleaned/preprocessed text. The numerical features are then used by the machine learning models and later by the deployed app pipeline.

vii. **ID:** P6

Depends on: [P1, P2, P3, P4, P5]

Used by: [EXP, P7, C0]

Short note: The machine learning models depend on the prior work for model choices, the dataset for training data, and preprocessing/feature extraction for usable input. These models are evaluated in experiments, and the selected model is deployed in the web app.

viii. **ID:** P7

Depends on: [P3, P4, P5, P6, EXP]

Used by: [C0]

Short note: The web application depends on the proposed workflow and the trained model pipeline. It also depends on the experimental result that identifies the Bagging Classifier as the model to deploy.

ix. **ID:** EXP

Depends on: [P2, P3, P4, P5, P6]

Used by: [P7, C0]

Short note: The experimental evaluation depends on the dataset, preprocessing, features, and machine learning models. Its results justify the claim that Bagging Classifier performs best by accuracy and support the final conclusion.

x. **ID:** C0

Depends on: [P0, P1, P2, P3, P4, P5, P6, P7, EXP]

Used by: []

Short note: The conclusion depends on the whole paper structure. It summarizes the problem, the model comparison, the web app, and future work.

Dependency Diagram of Paper Components

Evaluating Machine Learning Models for Sentiment Analysis of YouTube Comments and Creating an Accessible Web Application for Comment Analysis

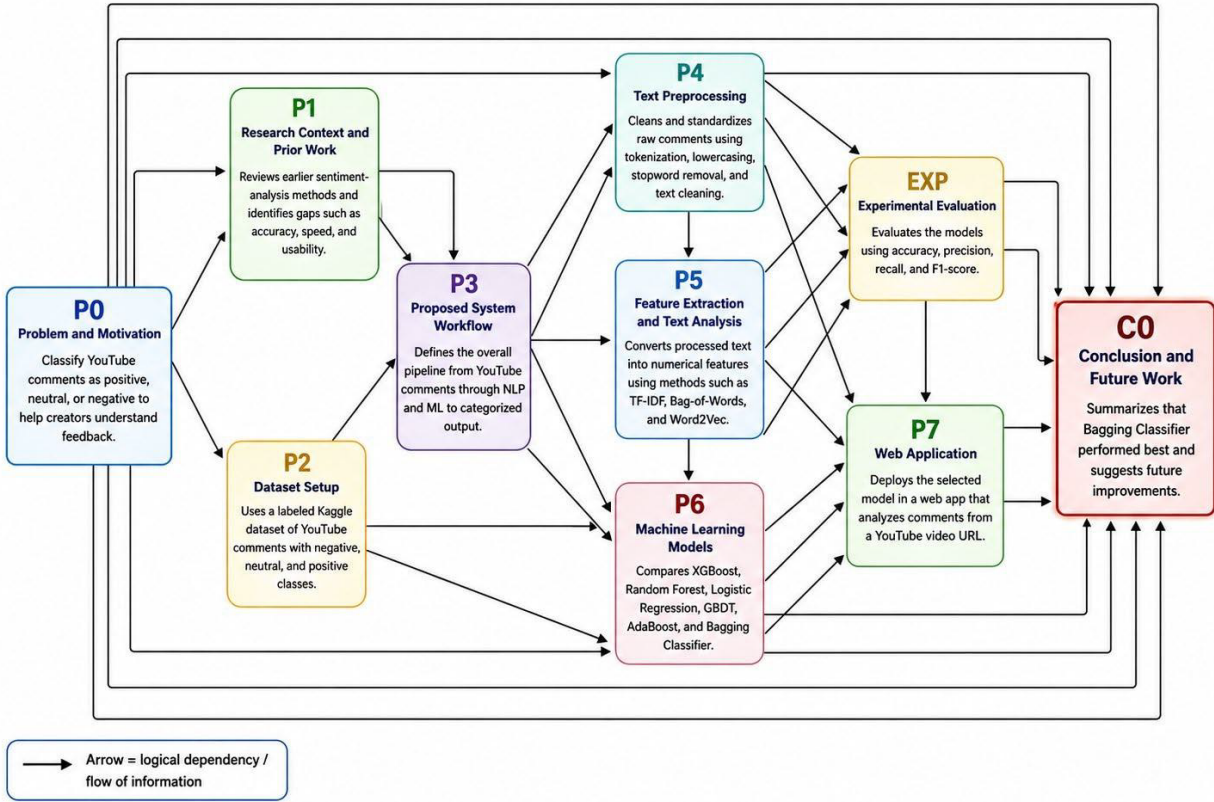


Figure 1. Dependency diagram

4.2 Directed Graph Structure

1. P0 -> P1
2. P0 -> P2
3. P0 -> P3
4. P0 -> P6

5. $P_0 \rightarrow P_7$
6. $P_0 \rightarrow \text{EXP}$
7. $P_0 \rightarrow C_0$
8. $P_1 \rightarrow P_3$
9. $P_1 \rightarrow P_6$
10. $P_1 \rightarrow P_7$
11. $P_1 \rightarrow \text{EXP}$
12. $P_1 \rightarrow C_0$
13. $P_2 \rightarrow P_4$
14. $P_2 \rightarrow P_5$
15. $P_2 \rightarrow P_6$
16. $P_2 \rightarrow \text{EXP}$
17. $P_2 \rightarrow C_0$
18. $P_3 \rightarrow P_4$
19. $P_3 \rightarrow P_5$
20. $P_3 \rightarrow P_6$
21. $P_3 \rightarrow P_7$
22. $P_3 \rightarrow \text{EXP}$
23. $P_3 \rightarrow C_0$
24. $P_4 \rightarrow P_5$
25. $P_4 \rightarrow P_6$
26. $P_4 \rightarrow \text{EXP}$
27. $P_4 \rightarrow P_7$
28. $P_4 \rightarrow C_0$
29. $P_5 \rightarrow P_6$
30. $P_5 \rightarrow \text{EXP}$

- 31. P5 -> P7
- 32. P5 -> C0
- 33. P6 -> EXP
- 34. P6 -> P7
- 35. P6 -> C0
- 36. EXP -> P7
- 37. EXP -> C0
- 38. P7 -> C0

5. Original Method Pipeline

5.1 Method Pipeline

Input → Step 1 → Step 2 → Step 3 → Step 4 → Step 5 → Output

Step	Step name	Inputs	Operation	Outputs
Input	YouTube comments	YouTube comments, including live/streaming comments	Comments are collected for sentiment analysis. In the web app, the user provides a YouTube video URL.	Raw comment text
Step 1	Text preprocessing	Raw YouTube comments	Standardizes the comments before analysis. The paper mentions tokenization, lowercasing, stopword removal, and text cleaning.	Cleaned/preprocessed comments
Step 2	Feature extraction	Cleaned/preprocessed comments	Converts text into numerical form so machine learning models can use it. The paper mentions TF- IDF, Bag-of- Words, and Word2Vec.	Numerical text features

Step 3	Text analysis / NLP processing	Preprocessed comments and/or numerical features	Uses NLP tools such as NLTK for tasks like tokenization, stemming, tagging, text classification, parsing, and semantic reasoning.	Processed text/features ready for model training
Step 4	ML model training and comparison	Processed features and sentiment labels	Trains or evaluates several ML models: XGB, RF, LR, GDBT, AdaBoost, and Bagging Classifier.	Trained sentiment classification models
Step 5	Model selection and deployment	Trained models and evaluation results	Selects the optimized trained model with favorable precision and high accuracy. The text says the deployed model uses the Bagging Classifier.	Deployed Bagging Classifier model
Output	Web application sentiment results	YouTube video URL / live comments + deployed model	The web app analyzes related comments and classifies them into three categories. It displays comments in tabular form, sentiment distribution, and changes over time.	Positive, negative, and neutral comment groups

5.2 Key Design Choices and Hyperparameters

Design choices clearly stated in the text

- a) **Task type:** Sentiment classification of YouTube comments.
- b) **Output classes:** Positive, negative, and neutral.
- c) **Preprocessing choices:** Tokenization, lowercasing, stopword removal, and text cleaning.
- d) **Text cleaning removes:** Punctuation, special characters, and numbers.
- e) **Feature extraction methods mentioned:** TF-IDF, Bag-of-Words, and Word2Vec.
- f) NLP toolkit mentioned: NLTK.
- g) **Other sentiment/NLP tools mentioned:** TextBlob and VADER.
- h) Machine learning models compared:
- i) XGBoost, Random Forest, Logistic Regression, Gradient Boosted Decision Tree, AdaBoost, and Bagging Classifier.
- j) **Deployment choice:** The Bagging Classifier is used in the web application.
- k) **Web app input:** YouTube video URL.
- l) **Web app output/display:** Tabular comment data, sentiment distribution, and sentiment changes over time.

Hyperparameters

- ✓ Specific model hyperparameters are **not clearly stated in the text**.
- ✓ Examples of missing hyperparameters include number of trees, learning rate, maximum depth, TF-IDF settings, Word2Vec dimensions, train/test split, and batch size. These are not clearly stated in the text.

5.3 Unclear or Vague Parts

- It is not clear from provided text which feature extraction method was actually used in the final experiments: TF-IDF, Bag-of-Words, Word2Vec, or a combination.
- It is not clear from provided text how the comments are collected from YouTube in the method section, except that the web app uses a YouTube video URL and gathers data from streaming YouTube video.
- It is not clear from provided text whether the system analyzes live comments only, existing video comments, or both.
- It is not clear from provided text how TextBlob and VADER are used. They are mentioned, but their exact role in the pipeline is not explained.
- It is not clear from provided text what “optimized trained model” means, because no optimization method or tuning process is described.
- It is not clear from provided text what training/testing split or validation method was used.
- It is not clear from provided text how the imbalanced dataset was handled.
- It is not clear from provided text what web framework or deployment method was used for the web application.

6. Original Experiments Summary

6.1 Experiment Matrix

- **Experiment_ID:** E1
Goal: Compare multiple machine learning models for YouTube comment sentiment classification.
Dataset(s): YouTube comments dataset; exact dataset details are not provided in this section.
Model/Variant: XGBoost, Random Forest, Logistic Regression, Gradient Boosted Decision Tree, AdaBoost, Bagging Classifier.
Baselines: The other tested ML models act as baselines against each other. No single official baseline is clearly named.
Metrics: Accuracy, precision, recall, F1-score.
Key result: Bagging Classifier has the highest accuracy: 0.9582 / 95.82%. Gradient Boosted Decision Tree has the highest precision: 0.9798 / 97.98%.
- **Experiment_ID:** E2
Goal: Evaluate the trained models using standard classification metrics. **Dataset(s):** Same YouTube comments dataset; not clearly described in this section. **Model/Variant:** All six trained classifiers: XGBoost, Random Forest, Logistic Regression, GBDT, AdaBoost, Bagging Classifier.
Baselines: Not clearly stated as separate baselines; comparison is among the six models.
Metrics: Accuracy, precision, recall, F1-score, based on TP, TN, FP, and FN.
Key result: Bagging Classifier performs best overall by accuracy and F1-score: accuracy = 0.9582, F1-score = 0.8802.
- **Experiment_ID:** E3
Goal: Compare the proposed method against previous state-of-the-art or earlier methods.
Dataset(s): Not clear from provided text whether all compared methods used the same dataset.
Model/Variant: Proposed Method, apparently based on Bagging Classifier.
Baselines: Musleh et al., Maw et al., Uddin et al., K.M. Kavitha et al., Cunha et al.
Metrics: Accuracy only.
Key result: Proposed Method reports 95.82% accuracy. It is higher than Musleh et al. at 94.62%, Uddin et al. at 92.88%, K.M. Kavitha et al. at 91.3%, and Cunha et al. at 60%–84%, but lower than Maw et al. at 97%.
- **Experiment_ID:** E4
Goal: Demonstrate the working of the developed web application.

Dataset(s): Comments captured through the Google API from YouTube.

Model/Variant: Deployed ML model; based on the earlier text, this appears to be the Bagging Classifier, but this specific section only says “ML model.” **Baselines:** None.

Metrics: No numerical metric is given for the web app demonstration.

Key result: The app classifies comments into positive, neutral, and negative groups and shows examples in figures.

6.2 Most Important Experiments

The most important experiment is E1, because it directly compares the six machine learning models and supports the paper’s main claim that the Bagging Classifier works best by accuracy.

The second most important experiment is E3, because it compares the proposed method with earlier methods. However, this comparison is weaker because the provided text does not clearly state whether the methods were tested on the same dataset or under the same conditions.

The web app demonstration, E4, is important for the application goal of the paper, but it is not a strong evaluation experiment because no performance numbers, user study, speed test, or usability test are provided.

6.3 Missing/Unclear Details

- The exact dataset name, size, class distribution, and source details are not clear from this section.
- The train/test split is not clearly stated.
- It is not clear whether cross-validation was used.
- It is not clear whether the reported results are training results, test results, or validation results. The text says “training performances,” which makes this confusing.
- It is not clear what preprocessing and feature extraction settings were used for each model.
- It is not clear whether TF-IDF, Bag-of-Words, Word2Vec, or a combination was used in the experiments.
- Model hyperparameters are not provided.
- It is not clear how the imbalanced dataset was handled.

- The comparison with previous methods is unclear because the datasets and experimental conditions may not be the same.
- The paper says the proposed system performs better than existing systems, but Table 3 shows Maw et al. = 97%, which is higher than the proposed method's 95.82%.
- In Section 7.2, the text says neutral remarks are separated but then "labels them as negative comments." This appears inconsistent, but the exact meaning is not clear from provided text.

7. Assumptions and Limitations

7.1 Explicit Assumption

- The paper assumes that YouTube comments can be grouped into three sentiment classes: positive, negative, and neutral. This is stated in the introduction and web app discussion.
- The paper assumes that YouTube comment analysis is useful for content creators because it can provide feedback on public video content and help identify trends, likes, dislikes, and neutral reactions.
- The paper assumes that manual comment analysis is difficult because YouTube comments contain a huge amount of information and a wide variety of comment types.
- The paper assumes that ML, DL, and NLP are suitable techniques for YouTube comment classification and linguistic feature extraction.
- The paper assumes that content creators face difficulty choosing among available YouTube comment analysis techniques, so experimental comparison is needed.
- The paper assumes that there is a need for more accurate and fast ML models for YouTube comment analysis.
- The paper assumes that a user-friendly web or mobile application is important for making ML-based YouTube comment analysis effective.
- The paper explicitly notes that imbalanced datasets are a future research issue, meaning the authors recognize class imbalance as something important to handle.

7.2 Inferred Assumptions

- The paper seems to assume that a Kaggle-trained model can be useful for live or online YouTube comments. This is implied because the model is trained experimentally and then deployed in a web app for live and online video analysis.
- The paper seems to assume that accuracy is the main criterion for selecting the best

model. Bagging Classifier is selected because it has the highest accuracy, even though Gradient Boosted Decision Tree has the highest precision.

- The paper seems to assume that the three-label sentiment setup is enough to capture useful creator feedback. It does not discuss more detailed emotions such as anger, sarcasm, humor, spam, or mixed sentiment.
- The paper seems to assume that comments collected through the Google API can be directly used as input to the ML model after preprocessing.
- The paper seems to assume that NLP preprocessing improves model performance, because the models are used along with NLP text preprocessing techniques.
- The paper seems to assume that a web interface makes the system more useful to content creators, because the paper emphasizes a user-friendly WebApp as one of its objectives.
- The paper seems to assume that model performance on the experimental data reflects usefulness in the deployed web app. However, the provided text does not clearly show a separate live-web-app evaluation.

7.3 Limitations and Open Problems

- The paper does not contain a separate “Limitations” section.
- The exact problem formulation is not formally stated in a separate section. The problem must be inferred from the introduction, objectives, and research gap text.
- The paper identifies multilingual comment analysis as future work. This means the current system may not fully handle comments in multiple languages.
- The paper identifies imbalanced datasets as future work. This suggests the current work may not fully solve class imbalance.
- The paper reports that Bagging Classifier has the best accuracy, but Gradient Boosted Decision Tree has the best precision. The trade-off between accuracy and precision is not deeply explained in the provided text.
- There is a possible inconsistency in the reported Bagging Classifier accuracy: the results section says 95.82%, while the conclusion says 95.88%.
- The comparison with existing systems may be limited because the provided text does

not clearly state whether all methods were tested on the same dataset, same metrics, or same experimental setup.

- The web app is demonstrated, but the provided text does not mention a usability study, user feedback, response time, deployment details, or real-time performance evaluation.
- Section 7.2 contains a confusing statement: it says the system separates neutral remarks but “labels them as negative comments.” This may be a writing error, but the exact meaning is not clear from provided text.

7.4 Items for Check

- Check whether the full paper gives the exact Kaggle dataset name, size, class distribution, and language.
- Check whether the paper explains the train/test split, validation method, or cross-validation.
- Check whether the paper states which feature extraction method was actually used: TF-IDF, Bag-of-Words, Word2Vec, or a combination.
- Check whether the paper gives model hyperparameters for XGB, RF, LR, GBDT, AdaBoost, and Bagging Classifier.
- Check whether the paper actually handles the imbalanced dataset, or only mentions it as future work.
- Check whether the web app analyzes live comments, existing video comments, or both.
- Check whether the comparison with previous work is fair, especially because one previous method reports 97% accuracy, which is higher than the proposed method’s 95.82%.
- Check the accuracy inconsistency: 95.82% vs. 95.88%.
- Check whether the “neutral remarks” sentence in Section 7.2 is a typo or a real labeling problem.

8. Critical Analysis and Selected Gap

8.1 Strengths

Methodological Strengths

- **Clear applied problem:**
The paper focuses on a practical problem: helping YouTube content creators understand whether audience comments are positive, neutral, or negative. This gives the work a clear real-world motivation.
- **End-to-end pipeline:**
The method is not only a classifier. It includes comment input, preprocessing, feature extraction, model training/comparison, model selection, and deployment in a web application.
- **Uses common NLP preprocessing steps:**
The paper mentions tokenization, lowercasing, stopword removal, and text cleaning. These are standard early steps for preparing raw text for machine learning.
- **Compares multiple machine learning models:**
The paper compares XGBoost, Random Forest, Logistic Regression, Gradient Boosted Decision Tree, AdaBoost, and Bagging Classifier. This is stronger than testing only one model.
- **Includes deployment as part of the work:**
The paper claims to deploy the selected model in a web application where users can enter a YouTube video URL. This supports the paper's goal of making sentiment analysis accessible to content creators.

Experimental Strengths

- **Uses several evaluation metrics:**
The experiments report accuracy, precision, recall, and F1-score. This is useful because accuracy alone may be misleading, especially if the dataset is imbalanced.
- **Reports model-specific results:**
The paper gives separate scores for each tested model. This makes it possible to compare the models directly.
- **Identifies different models as best under different metrics:**
Bagging Classifier has the best accuracy and F1-score, while Gradient Boosted Decision Tree has the best precision. This shows that “best model” depends on which metric matters most.

- **Includes comparison with earlier work:**

The paper compares the proposed method with several previous studies. This helps place the proposed method in a wider research context, although the fairness of that comparison is unclear.

8.2 Weaknesses / Risks

Assumption-Based Risks

- **The three-class sentiment setup may be too simple:**

The paper assumes comments can be usefully grouped into positive, neutral, and negative. However, YouTube comments may include sarcasm, spam, hate speech, mixed sentiment, slang, emojis, or multilingual text.

- **Kaggle-to-live YouTube transfer is not clearly justified:**

The paper trains and evaluates using a Kaggle dataset, but the web application is meant for live or online YouTube comments. It is not clear whether performance on the Kaggle dataset transfers well to real comments collected through the YouTube API.

- **Accuracy seems to drive model selection:**

Bagging Classifier is chosen because it has the highest accuracy, but Gradient Boosted Decision Tree has the highest precision. The paper does not clearly explain why accuracy should matter more than precision, recall, or F1-score for this application.

- **The usefulness of the web app is assumed more than tested:**

The paper says the web app is user-friendly, but the provided text does not mention a usability study, user feedback, accessibility evaluation, or interface performance test.

Method and Reproducibility Risks

- **Dataset details are missing:**

The exact Kaggle dataset name, size, language, class distribution, and source details are not clear from the provided text.

- **Feature extraction choice is unclear:**

The paper mentions TF-IDF, Bag-of-Words, and Word2Vec, but it is not clear which method was actually used in the final experiments.

- **TextBlob and VADER are not clearly explained:**

These tools are mentioned, but their exact role is unclear. They may be used for sentiment analysis, labeling, preprocessing, or comparison, but the provided text does not

specify.

- **Model hyperparameters are missing:**

The paper does not clearly state settings such as number of trees, learning rate, max depth, regularization, TF-IDF settings, or Word2Vec dimensions.

- **Train/test setup is unclear:**

It is not clear whether the reported results are from training, validation, testing, or cross-validation. This is especially important because the experiment section refers to “training performances.”

- **Handling of imbalance is unclear:**

The paper recognizes imbalanced datasets as future work, but it does not clearly explain how imbalance is handled in the current experiments.

Experimental Risks

- **Comparison with previous work may not be fair:**

Table 3 compares the proposed method with earlier systems, but it is not clear whether all methods used the same dataset, same labels, same metrics, and same evaluation setup.

- **The claim of outperforming existing systems is questionable:**

The proposed method reports 95.82% accuracy, but Maw et al. is listed with 97% accuracy. So the provided comparison does not fully support the claim that the proposed system performs better than all existing systems considered.

- **Accuracy value is inconsistent:**

The results section reports 95.82%, while the conclusion reports 95.88%. This may be a small typo, but it weakens reporting consistency.

- **Web app evaluation lacks numerical evidence:**

The web app is demonstrated with figures, but there are no clear metrics for speed, real-time performance, API reliability, user experience, or classification quality on live comments.

- **Possible labeling confusion in Section 7.2:**

The text says neutral remarks are separated but then “labels them as negative comments.” This may be a writing mistake, but it creates uncertainty about the output labels.

8.3 Questions and What-Ifs

- **What if the Kaggle dataset is very different from real YouTube comments?**

The model may perform well in experiments but worse when deployed on live or recent YouTube comments.

- **What if the dataset is imbalanced?**

High accuracy may mainly reflect good performance on the majority class. Per-class precision, recall, F1-score, and a confusion matrix would be important.

- **What if neutral comments are the hardest class?**
The provided results do not show class-wise performance, so it is unclear whether the model handles neutral comments well.
- **What if content creators care more about negative comments than overall accuracy?**
A model with the highest accuracy may not be the best if the real goal is to detect negative feedback reliably.
- **What if precision matters more than accuracy?**
Gradient Boosted Decision Tree has the highest precision, so choosing Bagging Classifier only because of accuracy may not always be the best decision.
- **What if comments contain sarcasm, slang, emojis, or mixed language?**
The provided method does not clearly explain how these cases are handled.
- **What if the web app must work in real time?**
The paper does not clearly report latency, throughput, API limits, or processing speed.
- **What if the previous studies used different datasets?**
Then the comparison in Table 3 would not prove that the proposed method is better. It would only show that the reported accuracy is competitive.
- **What if the feature extraction method changes the result strongly?**
Since TF-IDF, Bag-of-Words, and Word2Vec are all mentioned, the final performance may depend heavily on which representation was actually used.

8.4 Items Requiring Human Judgment

- Whether the paper's main contribution should be viewed as a **model comparison**, a **web application**, or the combination of both.
- Whether **accuracy** is the right main metric for this task, especially given the possible dataset imbalance.
- Whether the Bagging Classifier should be considered the best model overall, since GBDT has higher precision.
- Whether the comparison with previous works is meaningful without knowing whether the same dataset and evaluation setup were used.
- Whether the paper provides enough detail for reproducibility.

- Whether the claim of analyzing “live YouTube comments” is fully supported by the experiments.
- Whether the web app is truly “user-friendly” or “accessible,” since no user study or accessibility evaluation is provided.
- Whether the 95.82% vs. 95.88% accuracy mismatch is only a typo or a more serious reporting issue.
- Whether the Section 7.2 statement about neutral remarks being labeled as negative comments is a writing error or a real classification/output problem.

8.5 Selected Gap

Gap Type: Missing analysis / missing interpretability extension

Gap Statement: The original paper classifies YouTube comments into sentiment categories, but it does not deeply explain what themes or topics appear inside those sentiment groups.

Why this gap is suitable: This gap is easy to test using TF-IDF keywords and bigram analysis. It does not require training a large model or collecting a huge new dataset.

9. Proposed Extension

9.1 Selected Gap

The original paper classifies YouTube comments into positive, neutral, and negative categories. However, the paper does not deeply explain what important words or common phrases appear inside each sentiment group. Because of this, the sentiment result tells us whether comments are positive, neutral, or negative, but it does not fully explain what people are talking about inside those groups.

9.2 Chosen Extension

The chosen extension is: TF-IDF Keyword Summary + Bigram Analysis for Each Sentiment Group

This extension will analyze the comments after they are grouped by sentiment label. For each sentiment group, positive, neutral, and negative, we will extract important TF-IDF keywords and common bigrams.

9.3 Why This Extension Was Chosen

This extension was chosen because it is directly connected to the original paper. The original paper already works with YouTube comments, NLP preprocessing, and sentiment categories. This extension does not change the main goal of the paper; instead, it adds an interpretability layer.

The extension is also realistic for a student project because it does not require collecting new data, building a new web application, or training a large model. It can be completed using Python in Kaggle with simple text-mining techniques.

9.4 Core Change

Original Paper:

YouTube comments are classified into positive, neutral, and negative categories.

Our Extension:

After comments are classified or labeled as positive, neutral, and negative, we extract:

- ✓ top TF-IDF keywords for each sentiment group
- ✓ top bigrams for each sentiment group
- ✓ simple theme summaries based on those keywords and bigrams

9.5 Expected Output

The expected outputs of our extension are:

- 1) Top TF-IDF keywords for positive comments.
- 2) Top TF-IDF keywords for neutral comments.
- 3) Top TF-IDF keywords for negative comments.
- 4) Top bigrams for positive comments.
- 5) Top bigrams for neutral comments.
- 6) Top bigrams for negative comments.
- 7) Bar charts showing important keywords and bigrams.
- 8) A short theme summary explaining what each sentiment group talks about.

9.6 Feasibility and Scope Plan

This extension is feasible because it uses the existing sentiment labels from the Kaggle dataset and applies lightweight text-mining methods. The work can be completed in a notebook environment without training a new large model or building a new web application.

10. Implementation - Our Work

10.1 Implementation Overview

Our implementation focuses on adding an interpretability layer to the original YouTube comment sentiment analysis work. The original paper classifies YouTube comments into positive, neutral, and negative sentiment categories. Our work does not replace that original task. Instead, our work analyzes each sentiment group separately to understand what important words and common two-word phrases appear inside each group.

The selected extension is: TF-IDF Keyword Summary + Bigram Analysis for Each Sentiment Group.

The main purpose of this implementation is to make the sentiment analysis more explainable. Instead of only knowing whether comments are positive, neutral, or negative, this extension helps identify what words and phrases are common in each sentiment group.

10.2 Dataset Used

The original paper states that it used a Kaggle dataset for YouTube comment sentiment analysis. The paper also describes the sentiment labels as 0 = negative, 1 = neutral, and 2 = positive.

For our implementation, we used the Kaggle YouTube Statistics dataset. The dataset contains two files: videos-stats.csv and comments.csv. For this extension, we used only comments.csv because it contains the comment text and sentiment labels required for our analysis.

Table 10.1: Dataset files and project use

File Name	Purpose in This Project
videos-stats.csv	Not used in the extension. This file contains video-level statistics such as title, likes, views, keyword, comments, and published date.
comments.csv	Used in the extension. This file contains comment text and sentiment labels.

The comments.csv file contains the following relevant columns:

Video ID: the video identifier

Comment: the comment text

Likes: the number of likes the comment received

Sentiment: the sentiment label of the comment

The main columns used in our implementation were Comment and Sentiment. The Comment column was used as the text input. The Sentiment column was used to divide comments into positive, neutral, and negative groups.

Table 10.2: Sentiment label mapping

Sentiment Value	Mapped Sentiment Group
0	negative
1	neutral
2	positive

After mapping the sentiment labels, the initial sentiment group counts from the generated `sentiment_group_counts.csv` file were:

Table 10.3: Initial sentiment group counts

Sentiment Group	Number of Comments
positive	11,432
neutral	4,638
negative	2,338
Total	18,408

After text cleaning and removal of empty cleaned comments, the `cleaned_comments.csv` file contained 18,118 comments. The cleaned dataset contained 11,375 positive comments, 4,408 neutral comments, and 2,335 negative comments.

10.3 Implementation Environment

The implementation was completed in Kaggle Notebook using Python. Kaggle was used because the dataset and code can be handled in the same online environment.

The main Python libraries used were:

- a) pandas: for loading and processing the dataset
- b) numpy: for numerical operations
- c) re: for text cleaning using regular expressions
- d) scikit-learn: for TF-IDF keyword and bigram extraction matplotlib: for creating visualizations
- e) os and zipfile: for saving and organizing output files

10.4 Preprocessing

Before applying TF-IDF and bigram analysis, the comment text was cleaned. The preprocessing steps were:

1. Convert all text to lowercase.
2. Remove URLs.

3. Remove punctuation.
4. Remove numbers and special characters.
5. Remove extra spaces.
6. Remove English stopwords.
7. Remove very short words.
8. Create a cleaned_text column.

These steps were used to reduce noise in the text before extracting keywords and bigrams.

10.5 Sentiment Grouping

The numeric sentiment labels were mapped into readable sentiment groups:

0 was mapped to negative

1 was mapped to neutral

2 was mapped to positive

After this mapping, comments were divided into three groups: positive, neutral, and negative.

This grouping was necessary because our extension analyzes each sentiment group separately.

10.6 TF-IDF Keyword Extraction

After preprocessing and sentiment grouping, TF-IDF keyword extraction was applied separately to positive, neutral, and negative comments.

The steps were:

1. Select all positive comments.
2. Apply TF-IDF to the cleaned text of positive comments.
3. Extract the top TF-IDF keywords for positive comments.
4. Repeat the same process for neutral comments.
5. Repeat the same process for negative comments.

This step produced keyword tables and bar charts for each sentiment group. Output files from this step:

- ✓ tfidf_keywords_positive.csv
- ✓ tfidf_keywords_neutral.csv
- ✓ tfidf_keywords_negative.csv
- ✓ tfidf_keywords_positive.png
- ✓ tfidf_keywords_neutral.png
- ✓ tfidf_keywords_negative.png

10.7 Bigram Analysis

After TF-IDF keyword extraction, bigram analysis was applied separately to each sentiment group. A bigram is a two-word phrase. Bigram analysis was included because two-word phrases can sometimes explain meaning better than single words.

The steps were:

1. Select all positive comments.
2. Extract top bigrams from positive comments.
3. Select all neutral comments.
4. Extract top bigrams from neutral comments.
5. Select all negative comments.
6. Extract top bigrams from negative comments.

This step produced bigram tables and bar charts for each sentiment group. Output files from this step:

- ✓ bigrams_positive.csv
- ✓ bigrams_neutral.csv
- ✓ bigrams_negative.csv
- ✓ bigrams_positive.png
- ✓ bigrams_neutral.png
- ✓ bigrams_negative.png

10.8 Theme Summary Table

After extracting TF-IDF keywords and bigrams, a summary table was created for the three sentiment groups. This table includes the top keywords and top bigrams for each sentiment group.

The output file was sentiment_theme_summary.csv. The table was used to manually interpret the main themes in positive, neutral, and negative comments.

10.9 Difference Between Original Paper and Our Work

Original Paper: The original paper focuses on classifying YouTube comments into positive, neutral, and negative categories using machine learning models. It also deploys the selected model in a web application.

Our Work: Our work focuses on interpretation after sentiment grouping. We used TF-IDF keyword extraction and bigram analysis to understand what important words and phrases appear inside each sentiment group. This makes the sentiment analysis more explainable.

11. Results - Our Work

11.1 Sentiment Group Distribution

The dataset was divided into three sentiment groups: positive, neutral, and negative. initial mapped dataset contained 18,408 comments; After removing empty cleaned comments, TF-IDF and bigram analysis were performed on 18,118 comments.

Table 11.1:

Sentiment Group	Number of Comments	Percentage
Positive	11,432	62.10%
Neutral	4,638	25.20%
Negative	2,338	12.70%
Total	18,408	100.00%

The distribution shows that the dataset is imbalanced. Positive comments are the largest group, while negative comments are the smallest group.

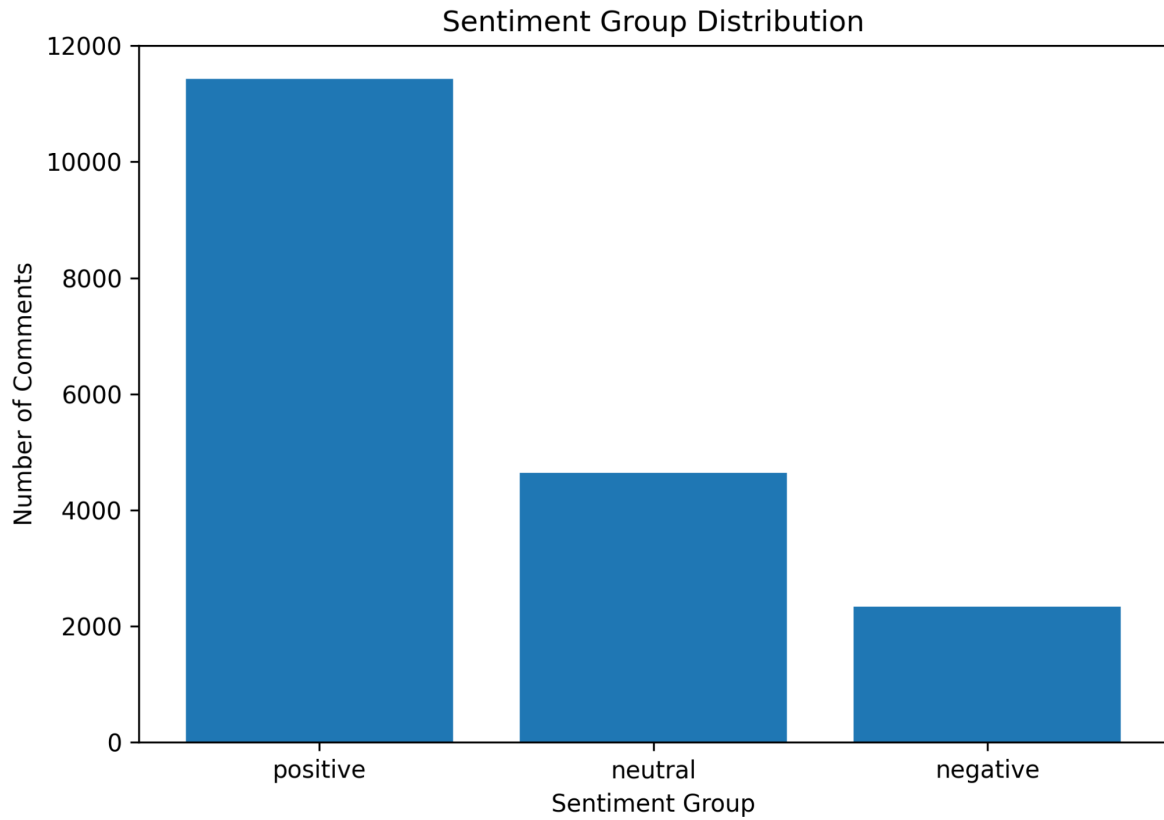


Figure 2. Sentiment group distribution.

TF-IDF Keyword Results

TF-IDF keyword extraction was applied separately to positive, neutral, and negative comments. The goal was to identify important individual words inside each sentiment group. Tables show top 10 values for readability; charts display the top 20 values.

Positive Comments - TF-IDF Keywords

Table 11.2:

Rank	Keyword	TF-IDF Score
1	love	0.031640
2	thank	0.021591
3	video	0.021006
4	great	0.020589
5	just	0.019311
6	like	0.019124

7	good	0.018328
8	really	0.016678
9	videos	0.016026
10	thanks	0.013852

From the output, the strongest positive keywords include love, thank, video, great, just, like, good. These terms are praise/appreciation terms plus topic keywords like data science and machine learning.

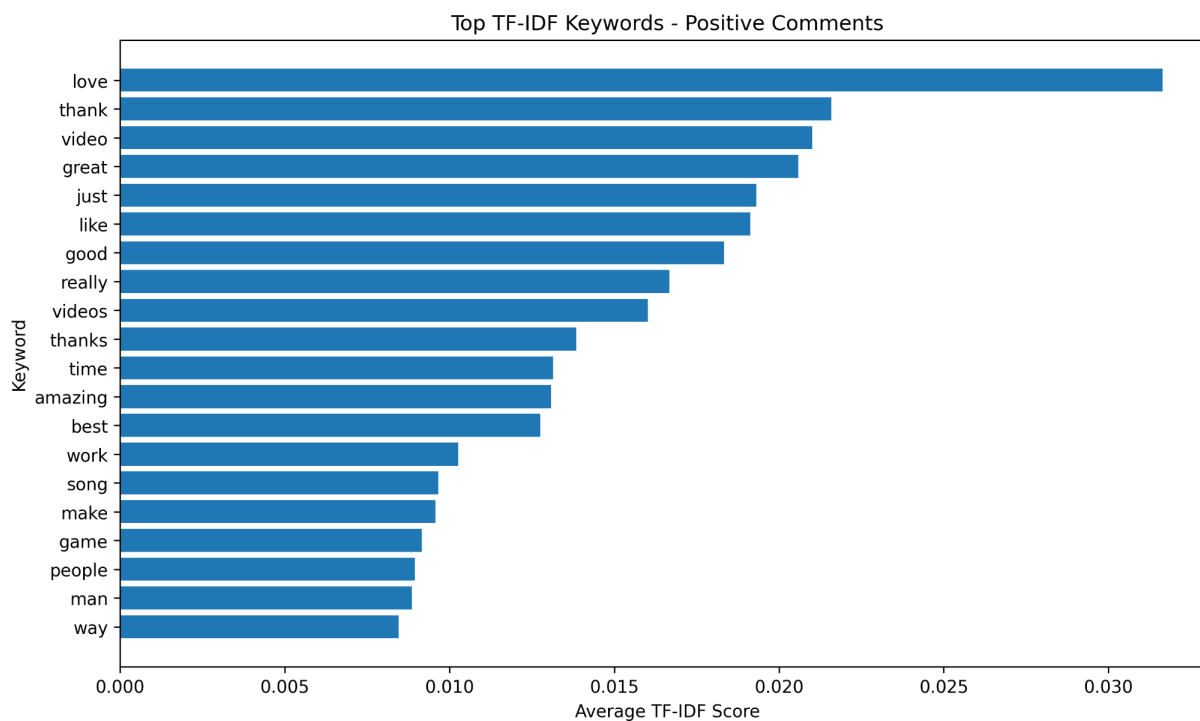


Figure 3. Top TF-IDF keywords - Positive comments.

Neutral Comments - TF-IDF Keywords

Table 11.3:

Rank	Keyword	TF-IDF Score
1	like	0.017414
2	que	0.013917
3	just	0.013616
4	video	0.012667

5	time	0.009995
6	know	0.009136
7	don	0.008898
8	make	0.008323
9	people	0.007694
10	think	0.007476

From the output, the strongest neutral keywords include **like, que, just, video, time, know, don**. These terms appear more general and less clearly emotional than the positive group.

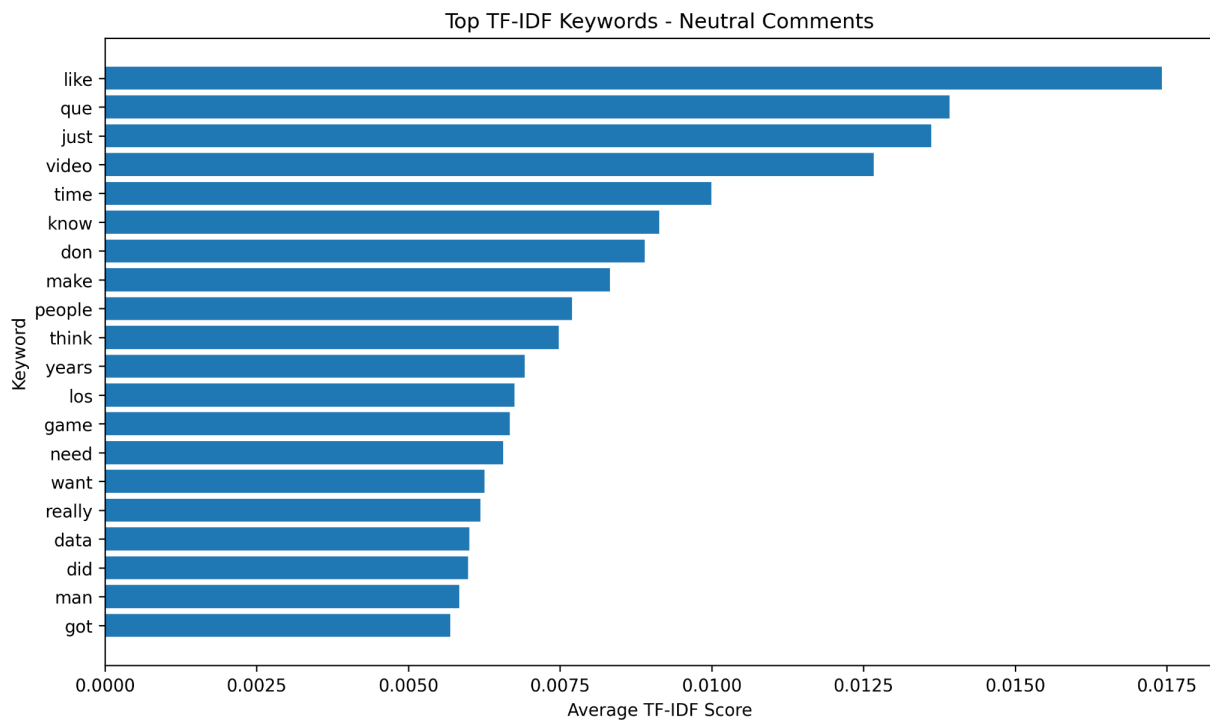


Figure 4. Top TF-IDF keywords - Neutral comments.

Negative Comments - TF-IDF Keywords

Table 11.4:

Rank	Keyword	TF-IDF Score
1	like	0.020467
2	just	0.020186
3	people	0.019417
4	don	0.017345
5	know	0.012560
6	time	0.012515
7	really	0.010379
8	think	0.009739
9	make	0.009134
10	got	0.009130

From the output, the strongest negative keywords include **like, just, people, don, know, time, really**. These words suggest uncertainty, disagreement, or negative reaction, but full comment context is still needed for exact interpretation.

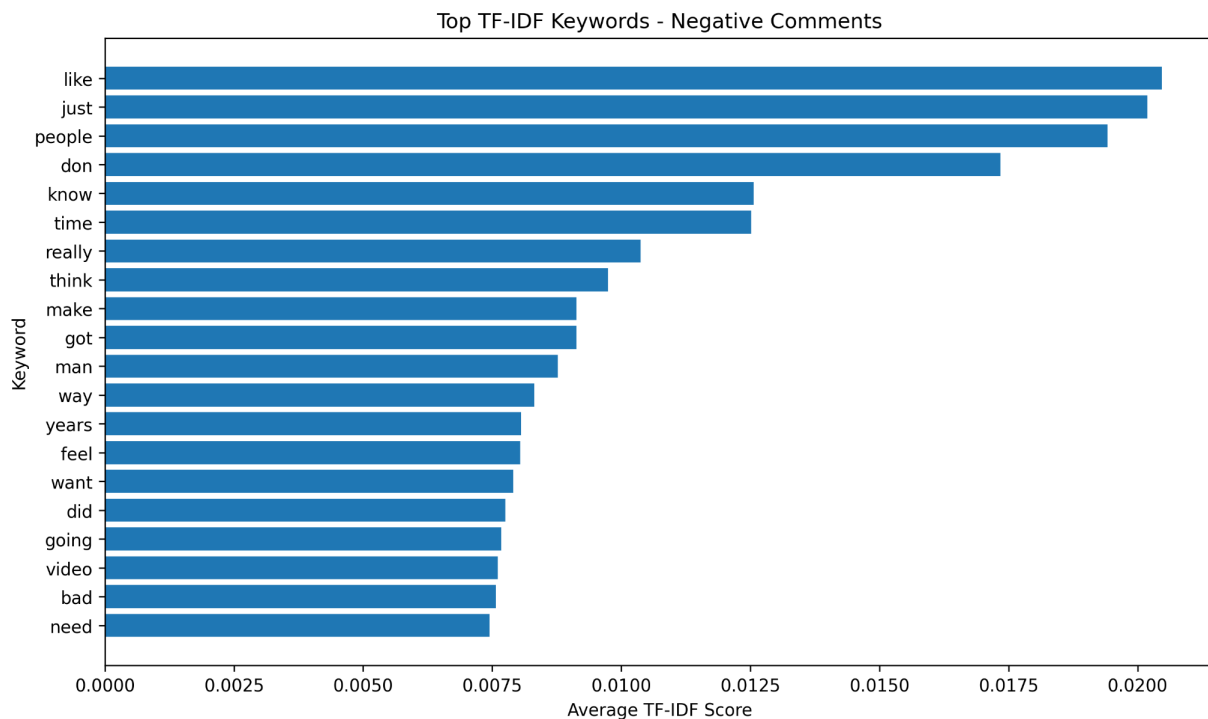


Figure 5. Top TF-IDF keywords - Negative comments.

11.2 Bigram Results

Bigram analysis was applied separately to each sentiment group. A bigram is a two-word phrase. This analysis helps show repeated phrases that may be more meaningful than individual keywords. Tables show top 10 values for readability; charts display the top 20 values.

Positive Comments - Top Bigrams

Table 11.5:

Rank	Bigram	TF-IDF Score
1	great video	0.005391
2	data science	0.004516
3	good work	0.004172
4	feel like	0.004118
5	love videos	0.003917
6	really good	0.003905
7	machine learning	0.003561
8	looking forward	0.003363
9	great job	0.003362
10	computer science	0.002828

The positive bigram output includes phrases such as **great video**, **data science**, **good work**, **feel like**, **love videos**, **really good**. These phrases show appreciation, praise, and positive engagement.

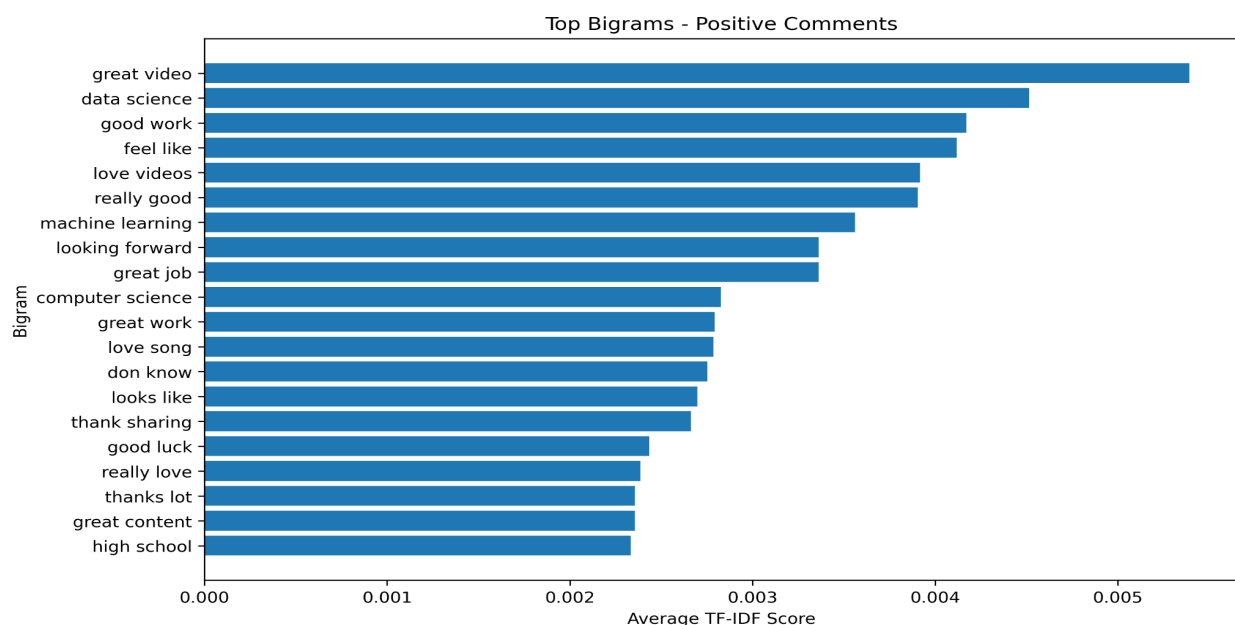


Figure 6. Top bigrams - Positive comments.

Neutral Comments - Top Bigrams

Table 11.6:

Rank	Bigram	TF-IDF Score
1	data science	0.003809
2	don know	0.003729
3	los mejores	0.003697
4	mejores conciertos	0.003607
5	feel like	0.003532
6	machine learning	0.003412
7	son unos	0.003157
8	unos los	0.003152
9	years ago	0.003088
10	fun fact	0.002750

The neutral bigram output includes phrases such as **data science**, **don't know**, **los mejores**, **mejores conciertos**, **feel like**, **machine learning**. The output also contains non-English phrases, which shows that the dataset includes mixed-language comments.

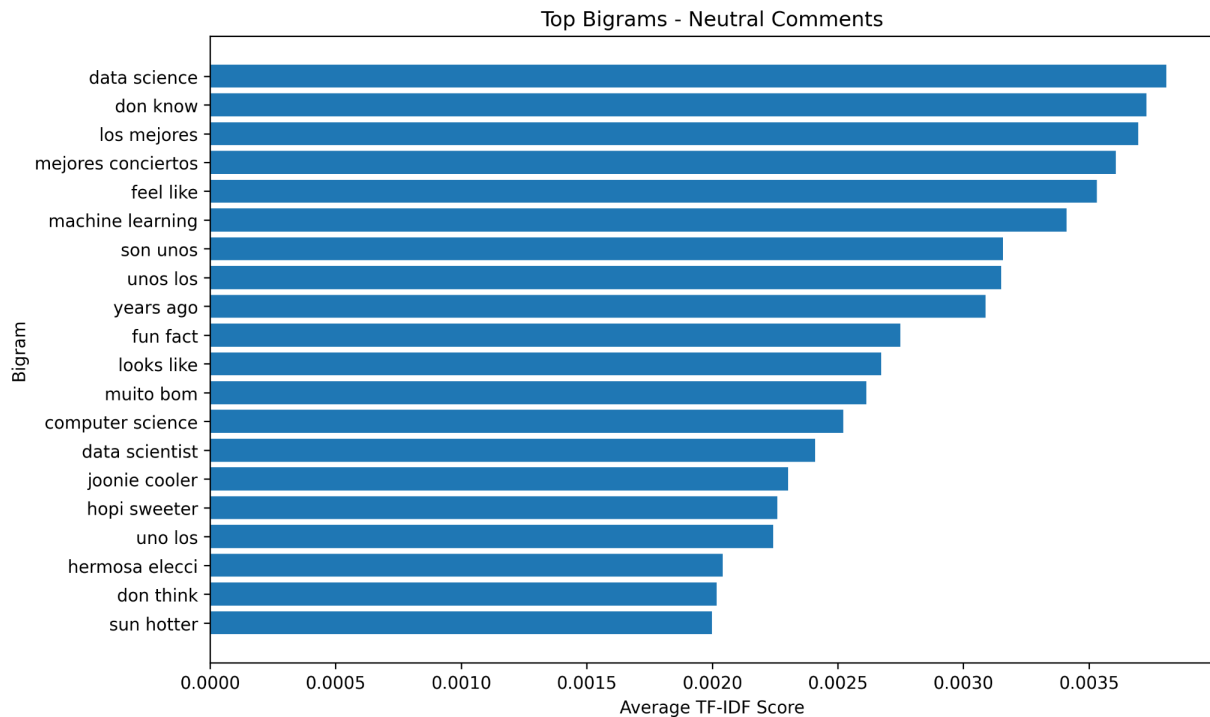


Figure 7. Top bigrams - Neutral comments.

Negative Comments - Top Bigrams

Table 11.7:

Rank	Bigram	TF-IDF Score
1	don know	0.006640
2	feel like	0.004731
3	sounds like	0.004008
4	years ago	0.003325
5	feel bad	0.002745
6	people like	0.002631
7	don think	0.002550
8	don want	0.002494
9	just like	0.002402
10	high school	0.002195

The negative bigram output includes phrases such as **don't know**, **feel like**, **sounds like**, **years ago**, **feel bad**, **people like**. These phrases give clearer clues about uncertainty, dissatisfaction, or negative reaction than single keywords alone.

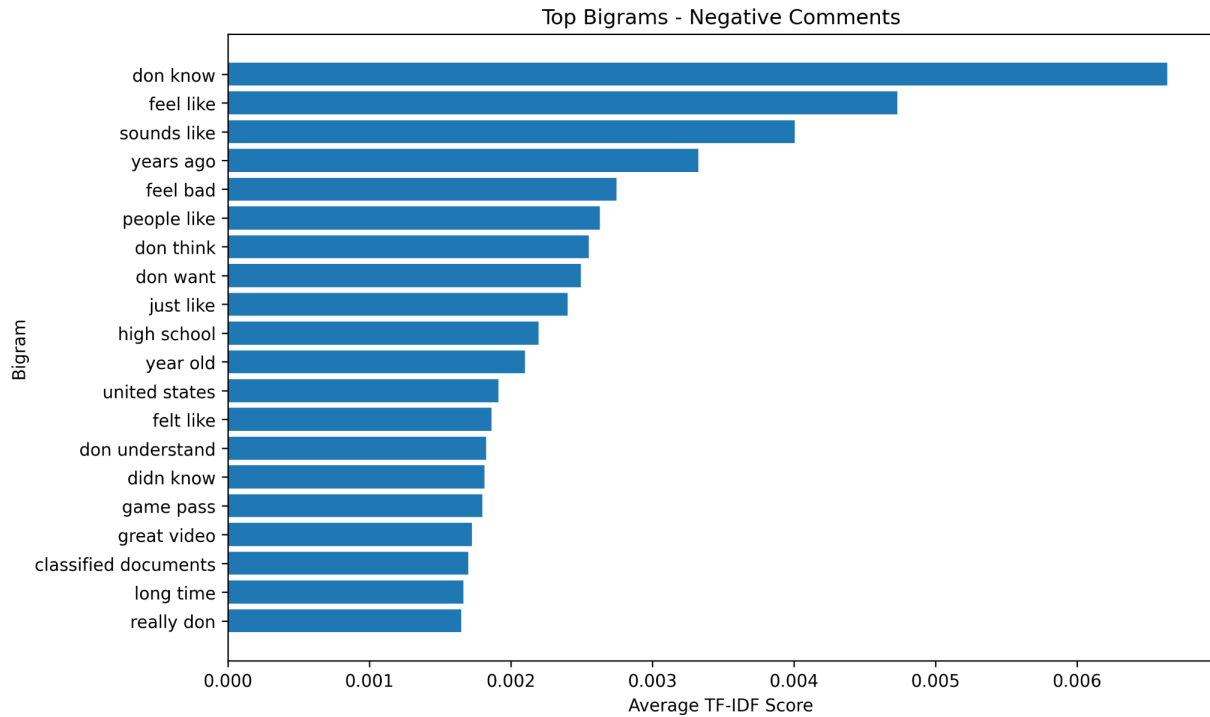


Figure 8. Top bigrams - Negative comments.

11.3 Sentiment Theme Summary

The final summary table was created from the output file `sentiment_theme_summary.csv`. The `possible_theme_label` column was intentionally left as “to be manually interpreted” in the generated output file, so no theme label is treated as original dataset data.

Table 11.8:

Sentiment Group	Top 5 TF-IDF Keywords	Top 5 Bigrams	Possible Theme Label
Positive	love, thank, video, great, just	great video, data science, good work, feel like, love videos	to be manually interpreted
Neutral	like, que, just, video, time	data science, don know, los mejores, mejores conciertos, feel like	to be manually interpreted
Negative	like, just, people, don, know	don know, feel like, sounds like, years ago, feel bad	to be manually interpreted

Manual interpretation based on the output: Positive comments contain strong

praise/appreciation terms such as love, thank, great video, and good work. Neutral comments contain general discussion terms and some mixed-language phrases. Negative comments contain phrases such as don know, feel bad, don think, and don want, which indicate uncertainty, dissatisfaction, or negative reaction. This interpretation is based only on the displayed TF-IDF and bigram outputs.

11.4 Result Summary

The results show that the three sentiment groups have different keyword and bigram patterns. Positive comments contain more appreciation-related terms, neutral comments contain more general and mixed- language discussion terms, and negative comments contain more uncertainty or dissatisfaction-related phrases.

This supports the selected extension. The original paper classifies comments into sentiment categories, while our extension helps explain what words and phrases appear inside those categories

12. Discussion

12.1 Purpose of the Discussion

The original paper classifies YouTube comments into positive, neutral, and negative categories, giving a general view of audience sentiment. Our extension improves interpretability by using TF-IDF keyword extraction and bigram analysis to identify important words and common phrases within each sentiment group.

Original Paper vs. Our Extension

Part	Focus
Original Paper	Classifies YouTube comments into positive, neutral, and negative groups using machine learning models.
Our Extension	Analyzes each sentiment group separately using TF-IDF keywords and bigrams to make the sentiment results more explainable.

Sentiment Group Distribution

From (Table:11.1) The dataset is imbalanced: positive comments dominate at 62.10%, neutral at 25.20%, and negative at 12.70%. This imbalance may affect the stability of keyword extraction for minority sentiment groups.

Interpretation of Positive Comments

Rank	TF-IDF Keyword	Keyword Score	Bigram	Bigram Score
1	love	0.031640	great video	0.005391
2	thank	0.021591	data science	0.004516
3	video	0.021006	good work	0.004172
4	great	0.020589	feel like	0.004118
5	just	0.019311	love videos	0.003917

The positive comments show clear appreciation-related patterns. The strongest TF-IDF keywords include **love, thank, video, great, just, like, good**. The strongest bigrams include **great video, data science, good work, feel like, love videos, really good, machine learning**. positive comments contain praise

terms plus topic terms such as data science and machine learning. This interpretation is based on the exact keyword and bigram outputs from the Kaggle notebook.

Interpretation of Neutral Comments

Rank	TF-IDF Keyword	Keyword Score	Bigram	Bigram Score
1	like	0.017414	data science	0.003809
2	que	0.013917	don know	0.003729
3	just	0.013616	los mejores	0.003697
4	video	0.012667	mejores conciertos	0.003607
5	time	0.009995	feel like	0.003532

The neutral comments appear more general and mixed. The top TF-IDF keywords include like, que, just, video, time, know, don. The top bigrams include data science, don know, los mejores, mejores conciertos, feel like, machine learning, son unos. These results suggest that the neutral group contains general discussion, technical terms, and some non-English or mixed-language phrases. Examples from the output include los mejores, mejores conciertos, son unos, and unos los.

Interpretation of Negative Comments

Rank	TF-IDF Keyword	Keyword Score	Bigram	Bigram Score
1	like	0.020467	don't know	0.006640
2	just	0.020186	feel like	0.004731
3	people	0.019417	sounds like	0.004008
4	don	0.017345	years ago	0.003325
5	know	0.012560	feel bad	0.002745

The negative comments show more uncertainty and dissatisfaction-related patterns. The top TF-IDF keywords include like, just, people, don, know, time, really. The top bigrams include don't know, feel like, sounds like, years ago, feel bad, people like, don't think.

12.2 What Bigram Analysis Added

The TF-IDF keyword results show important single words, but some single words are difficult to interpret alone. For example, words such as like, just, people, time, and know appear in multiple sentiment groups. These words are common and do not always explain the meaning clearly.

The bigram results provide better context because they show two-word phrases. For positive comments, phrases such as great video, good work, love videos, and great job provide direct positive meaning. For negative comments, phrases such as don know, feel bad, don think, and don want provide clearer evidence of uncertainty or dissatisfaction. Therefore, bigram analysis improved the interpretability of the sentiment groups.

12.3 Usefulness of the Extension

The original sentiment labels show whether a comment is positive, neutral, or negative, while the TF-IDF keyword output highlights important words within each sentiment group and the bigram analysis identifies common two-word phrases that are easier to interpret. These sentiment-wise keyword and bigram summaries help content creators better understand the reasons behind audience feedback, and the entire extension is simple enough to run in a Kaggle notebook without requiring a large model or web application.

12.4 Limitations of Our Work

1. The dataset is imbalanced. Positive comments are much more common than negative comments, so the positive keyword results may be more stable.
2. Some extracted words are still generic, such as like, just, video, and time. This shows that preprocessing can affect keyword quality.
3. Some neutral bigrams contain non-English or mixed-language phrases.
4. The English-only preprocessing does not fully handle multilingual comments.
5. The theme interpretation is manual. The output files provide keywords and bigrams, but the final meaning still requires human judgment.
6. TF-IDF and bigram analysis are simple text-mining methods. They may not fully capture sarcasm, slang, emojis, or deeper context.

12.5 Discussion Summary

Overall, the extension fills the gap by not only classifying comments into sentiment categories but also explaining the common words and phrases within each group. Positive comments mainly show appreciation, neutral comments contain general discussions, and negative comments reflect dissatisfaction, making the sentiment analysis more interpretable while remaining suitable for a small undergraduate project.

13. Conclusion

The original paper developed a machine-learning-based system for YouTube comment sentiment analysis. It compared several machine learning models and reported that the Bagging Classifier performed best in terms of accuracy. The original paper also deployed the selected model in a web application for YouTube comment analysis.

Our project extended the original work by adding TF-IDF keyword summary and bigram analysis for each sentiment group. Instead of only classifying comments as positive, neutral, or negative, our extension also identifies important words and common two-word phrases inside each group.

The implementation used the comments.csv file from the Kaggle YouTube Statistics dataset. The comments were grouped using the existing sentiment labels, where 0 represents negative, 1 represents neutral, and 2 represents positive. The initial mapped dataset contained 18,408 comments. After text cleaning and removal of empty cleaned comments, TF-IDF and bigram analysis were performed on 18,118 comments: 11,432 positive comments, 4,638 neutral comments, and 2,338 negative comments.

The results showed that positive comments contained appreciation-related words and phrases such as love, thank, great video, good work, and love videos. Neutral comments contained more general discussion and mixed-language phrases, including terms such as data science, machine learning, and los mejores. Negative comments contained phrases such as don know, feel bad, don think, and don want, which helped explain negative reactions more clearly.

Overall, this extension makes sentiment analysis more interpretable. It helps content creators understand not only the overall sentiment of the comments, but also the common words and phrases connected to each sentiment group. This makes the original sentiment analysis task more useful for understanding audience feedback.

Summary of Original Paper and Our Extension

Original Paper	Our Extension
Classified YouTube comments into positive, neutral, and negative categories.	Analyzed each sentiment group separately using TF-IDF keywords and bigrams.
Compared machine learning models and selected Bagging Classifier based on accuracy.	Used the existing sentiment labels and focused on interpretability instead of training a new large model.
Built a web application for YouTube comment sentiment analysis.	Generated tables, charts, and theme summaries to explain what appears inside each sentiment group.

Final statement: The project does not claim to replace the original paper. It adds a simple, realistic, and explainable text-mining layer on top of the original sentiment analysis task.

14. References

14.1 Original Research Paper

Shivsharan, N., Kambli, V., Dabholkar, S., Dalvi, A., C Sukali, T. (2025). Evaluating machine learning models for sentiment analysis of YouTube comments and creating an accessible web application for comment analysis. *Procedia Computer Science*, 258, 3165-3174.
<https://doi.org/10.1016/j.procs.2025.04.574>

14.2 Dataset

Patil, A. (2022). YouTube Statistics [Dataset]. Kaggle.
<https://www.kaggle.com/datasets/advaypatil/youtube-statistics>

15. Appendix

File Name	Use in This Project	Description
videos-stats.csv	Not used in the selected extension	Contains video-level information such as title, video ID, published date, keyword, likes, comments, and views.
comments.csv	Used	Contains the comment text, comment likes, video ID, and sentiment label.
Sentiment Value	Sentiment Group	
0	negative	
1	neutral	
2	positive	
Sentiment Group	Number of Comments	Percentage
Positive	11,432	62.10%
Neutral	4,638	25.20%
Negative	2,338	12.70%
Total	18,408	100.00%